

API integration: Uses axios and native fetch() to call backend REST APIs hosted on each microservice.

WebSocket connections: Enables real-time chat and live notifications from the backend via persistent sockets.

Session management: Local storage and Clerk's SDK manage secure token handling across sessions.

▪ **Authentication with Clerk.dev**

Authentication and authorisation are handled using Clerk.dev, a plug-and-play identity platform offering prebuilt sign-up/sign-in UIs, secure JWT-based session management, role-based access control (e.g., buyer, seller) and integration with the frontend through Clerk's React SDK.

Once users log in, Clerk provides JWTs that are attached to every frontend API request. Backend services validate these tokens to ensure only authenticated users can access protected routes.

This offloads complex auth logic from your codebase while keeping identity management secure and scalable.

Experimental challenges and learnings

Building this microservices-based e-commerce platform was not without its hurdles. From inter-service communication bugs to deployment quirks in Kubernetes, every step offered valuable hands-on experience. Given below are some of the key challenges faced and what we learned from resolving them.

▪ **Kafka setup and debugging**

Challenge: Setting up Kafka on Minikube and ensuring services would consistently produce/consume events.

Learning: Network policies in Kubernetes can block inter-pod communication if not configured properly. Exposing Kafka with correct listeners and using kafkajs with proper retry logic helped stabilise message flow.

▪ **WebSocket handling**

Challenge: Maintaining persistent WebSocket connections across multiple services and pods.

Learning: Centralising WebSocket logic in dedicated services (Chat, Notification) and using topics to forward events from Kafka to frontend greatly simplified architecture and improved stability.

▪ **Autoscaling with HPA**

Challenge: CPU thresholds were too sensitive, leading to unnecessary scaling.

Learning: Fine-tuning targetCPUUtilizationPercentage and using custom metrics made autoscaling more predictable.

▪ **Auth token propagation**

Challenge: Passing JWT tokens across services and

ensuring backend verification.

Learning: Each service validates tokens individually via middleware; Clerk's SDK made token decoding easier, but caching Clerk's public keys reduced verification latency.

▪ **MongoDB vs PostgreSQL use**

Challenge: Choosing when to use NoSQL or relational databases.

Learning: Use MongoDB for flexible and fast reads (e.g., product listings), and PostgreSQL when data consistency and transactions are critical (e.g., orders, payments).

Overall, these challenges transformed theoretical understanding into real DevOps and backend engineering skills—highlighting the power (and complexity) of distributed systems in real-world scenarios.

This project was a comprehensive exercise in building a real-world, production style application using a microservices architecture powered entirely by open source technologies. By combining tools like Node.js, Kafka, Docker, Kubernetes, MongoDB, and Next.js, we were able to create a scalable, modular, and reactive e-commerce platform that mirrors modern cloud-native applications.

Beyond technical implementation, the project provided deep insights into the challenges of distributed system design, inter-service communication, data consistency, authentication, and observability.

Future enhancements

Ingress controller: To enable external access and URL routing with TLS support.

CI/CD pipeline: Automating deployment with tools like GitHub Actions or Jenkins.

Observability tools: Integrate Prometheus and Grafana for real-time system monitoring.

Advanced caching: Use Redis to optimise product listings and reduce DB load.

Cloud deployment: Port the Minikube setup to a managed Kubernetes service like GKE or EKS.

With modular design, open source tooling, and hands-on implementation, this project not only served as a strong learning experience but also demonstrated the feasibility of building robust systems without relying on closed source or proprietary platforms. **END** 🐧

By: Vihan Vashishth, Vineet Priyadarshi and B. Thangaraju

The authors are associated with the Open Source Technology Lab at the International Institute of Information Technology, Bengaluru.